

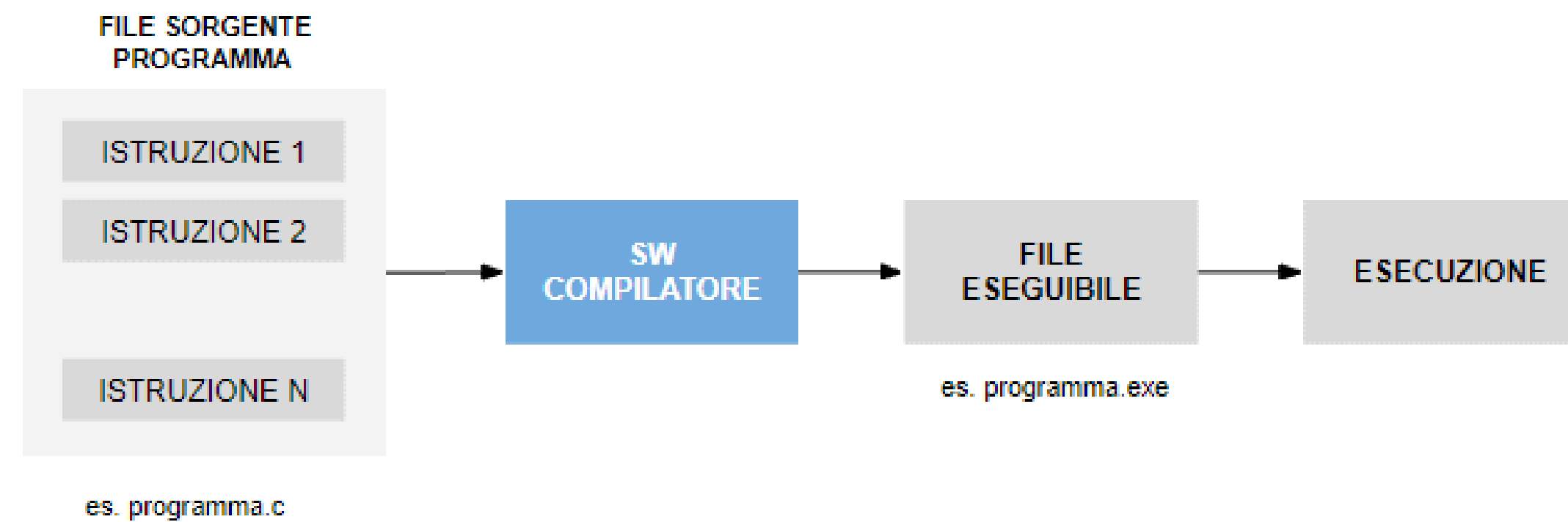
Eco-design Digitale di Base per i servizi ICT

Programmazione in C e Python

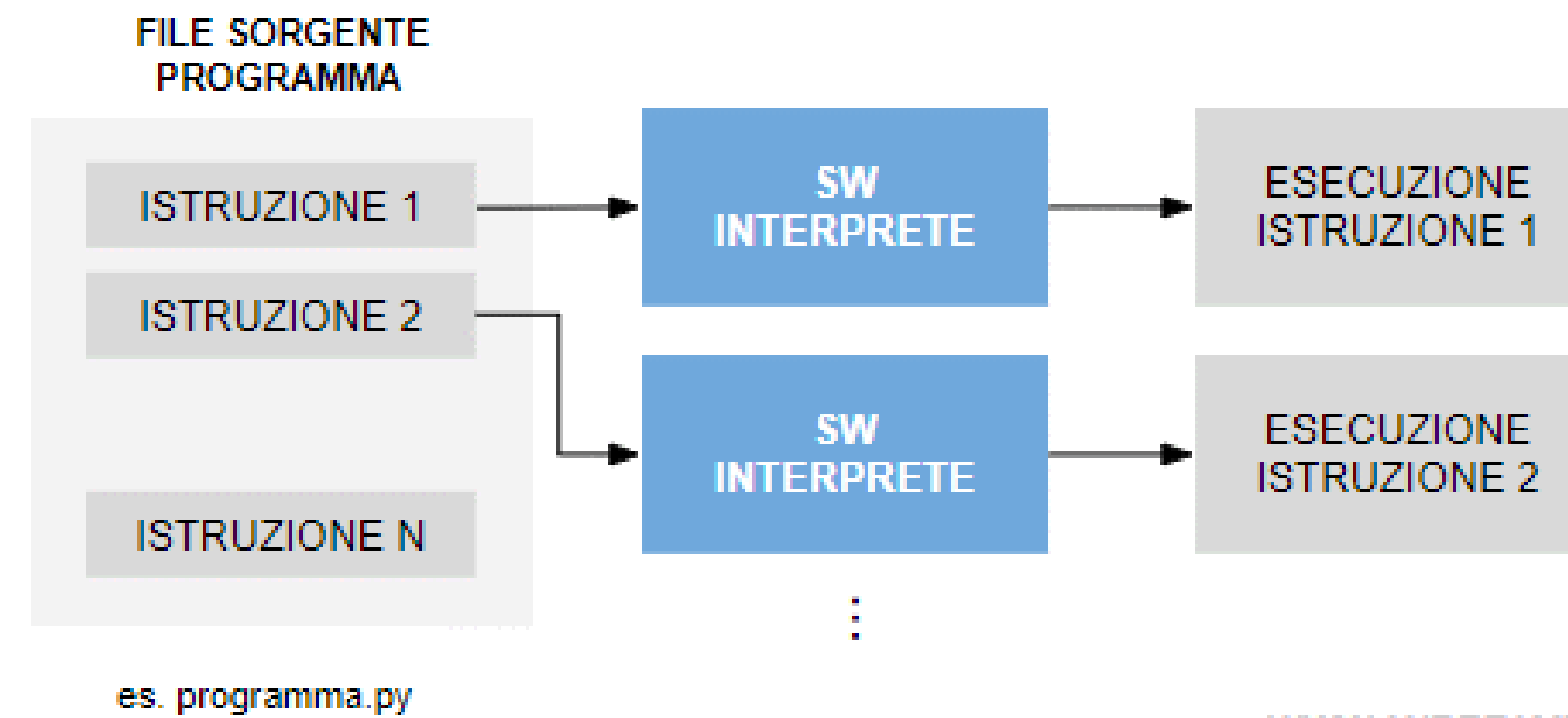
Obiettivi della lezione

- Acquisire familiarità con la sintassi e i tipi di dato di Python
- Comprendere e usare strutture di controllo (if, for, while)
- Scrivere funzioni e utilizzare moduli
- Introdurre lo scripting per automatizzare attività comuni
- Realizzare semplici analisi su file di testo o CSV

Compilato vs Interpretato



WWW.ANDREAMININI.COM



WWW.ANDREAMININI.COM

Linguaggio Python

```
33 self.logdupes = True
34 self.debug = debug
35 self.logger = logging.getLogger(__name__)
36
37 if path:
38     self.file = open(os.path.join(path, 'requests.log'), 'a')
39     self.file.seek(0)
40     self.fingerprints.update(request)
41
42 @classmethod
43 def from_settings(cls, settings):
44     debug = settings.DEBUG
45     return cls(job_dir(settings), debug)
46
47 def request_seen(self, request):
48     fp = self.request_fingerprint(request)
49     if fp in self.fingerprints:
50         return True
51     self.fingerprints.add(fp)
52     if self.file:
53         self.file.write(fp + os.linesep)
54
55 def request_fingerprint(self, request):
```

Linguaggio Python

- **Ad alto livello:** sintassi vicina al linguaggio naturale
- **Interpretato:** eseguito direttamente senza compilazione
- **Dinamico:** tipi di variabili determinati a runtime

Ambiti di utilizzo principali:

- Sviluppo web (Django, Flask)
- Data science e analisi dati
- Scripting e automazione
- Intelligenza artificiale e machine learning
- Sviluppo di applicazioni desktop
- **Perché Python?** Sintassi semplice, leggibile e compatta che favorisce la produttività.

Ciao mondo!

```
print("Hello world!")
```


Fasi di esecuzione di un programma

Dall'editing alla compilazione

1. **Editing:** il programmatore scrive il codice usando un editor (es. nano, o IDE come Visual Studio)

2. **Parsing:** Il file .py viene letto dall'interprete Python.

Il codice viene **analizzato sintatticamente** per verificare che la grammatica del linguaggio sia corretta.

3. **Compilazione in bytecode**

Il codice sorgente viene **compilato in bytecode**, una rappresentazione intermedia e ottimizzata.

Questo bytecode è salvato in file .pyc all'interno della cartella `__pycache__`.

Fasi di esecuzione di un programma

Dal linking al loading

4. Interpretazione del bytecode

Il bytecode viene **interpretato dalla Python Virtual Machine (PVM)**.

La PVM esegue istruzione per istruzione traducendo il bytecode in comandi macchina a runtime.

5. (Facoltativo) Import e caricamento moduli

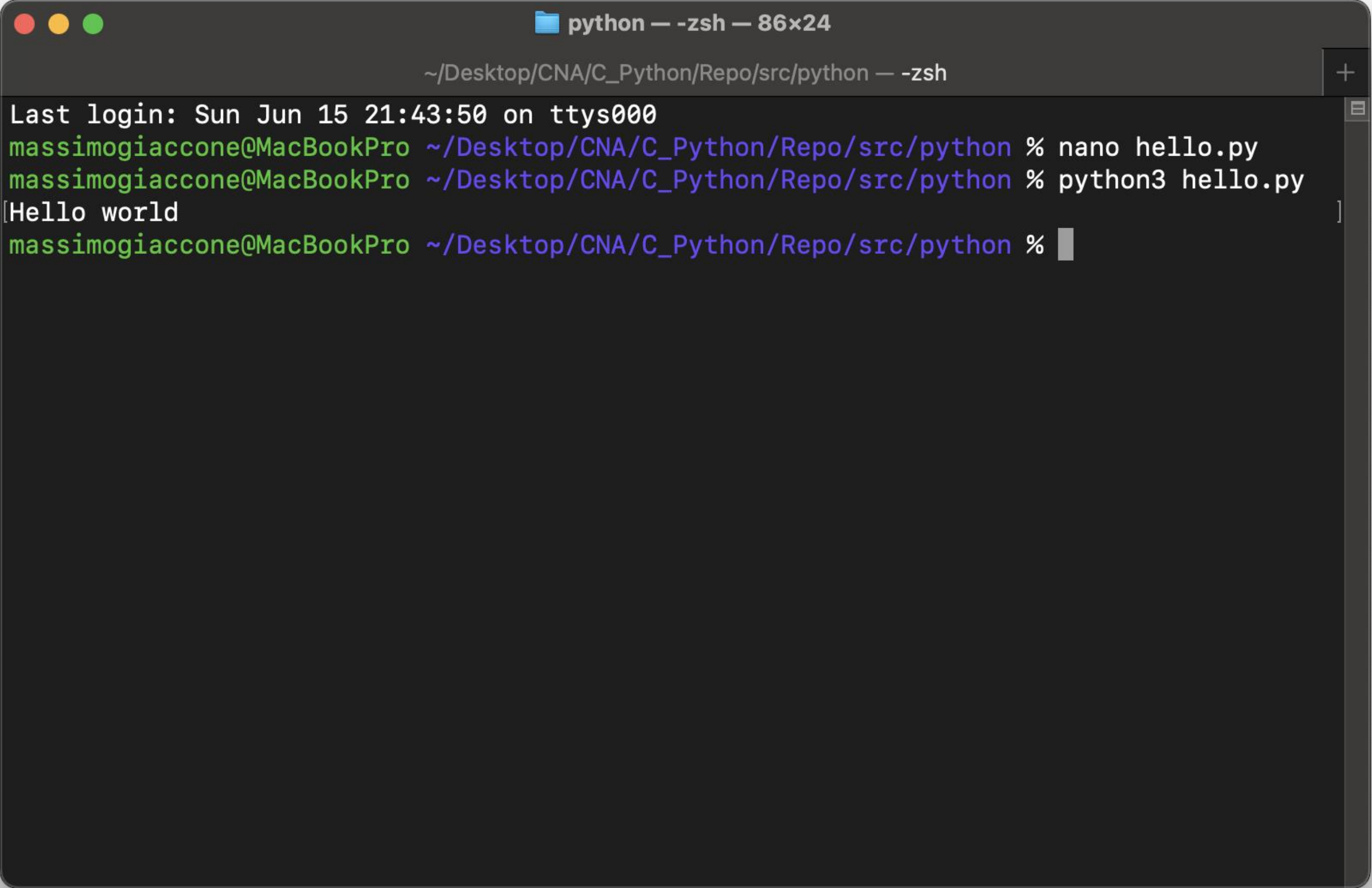
Se nel programma sono presenti import, i moduli vengono localizzati, caricati e compilati se necessario.

Fasi di esecuzione di un programma

Esecuzione

6. Esecuzione: la CPU esegue il programma, istruzione per istruzione

Esempio:

A terminal window titled "python — -zsh — 86x24" with a subtitle "~/Desktop/CNA/C_Python/Repo/src/python — -zsh". The terminal shows the following sequence of commands and output:

```
Last login: Sun Jun 15 21:43:50 on ttys000
massimogiaccone@MacBookPro ~/Desktop/CNA/C_Python/Repo/src/python % nano hello.py
massimogiaccone@MacBookPro ~/Desktop/CNA/C_Python/Repo/src/python % python3 hello.py
Hello world
massimogiaccone@MacBookPro ~/Desktop/CNA/C_Python/Repo/src/python %
```

Installiamo l'editor

Utenti Windows & Mac

- <https://www.python.org/>
Installare la versione più recente di Python (3.12.9 preferibile)
- Visual Studio Code

Come testiamo la PVM

1. Aprire Terminale/Prompt dei comandi
2. Digitare `python3 --version`
3. Creare un file con estensione `.py`
4. Lanciare il programma con il comando:
`python3 nome_del_file.py`

IDE/Compilatori online

- https://www.onlinegdb.com/online_python_compiler
- <https://replit.com/>

Lessico Python

Variabili e tipi di dato

- Variabili: locazione di memoria a cui viene associato un nome e che contiene un valore.
- **Tipi numerici:**
 - int: numeri interi (42, -10, 0)
 - float: numeri decimali (3.14, -2.5, 1.0)
- **Altri tipi fondamentali:**
 - str: stringhe di testo ("Ciao", 'Python')
 - bool: valori booleani (True, False)

Lessico Python

Variabili e tipi di dato

- Variabili: locazione di memoria a cui viene associato un nome e che contiene un valore.

Collezioni:

- list: liste ordinate e modificabili [1, 2, 3]
- tuple: tuple ordinate ma immutabili (1, 2, 3)
- set: insiemi non ordinati {1, 2, 3}
- dict: dizionari chiave-valore {"nome": "Mario"}

Lessico Python

Commenti

- I **commenti** servono a documentare il codice e non vengono eseguiti.

```
main.py
1  '''
2
3      Online Python Compiler.
4      Code, Compile, Run and Debug python program online.
5  Write your code in this editor and press "Run" button to execute it.
6  |
7  '''
8
9  name = input("Inserisci il tuo nome: ")
10 # f-string
11 print(f"Il tuo nome è {name}")
12 print("Ciao " + name)
13
```



*È buona prassi commentare
il codice in maniera da
renderlo più leggibile*

Lessico Python

Costanti

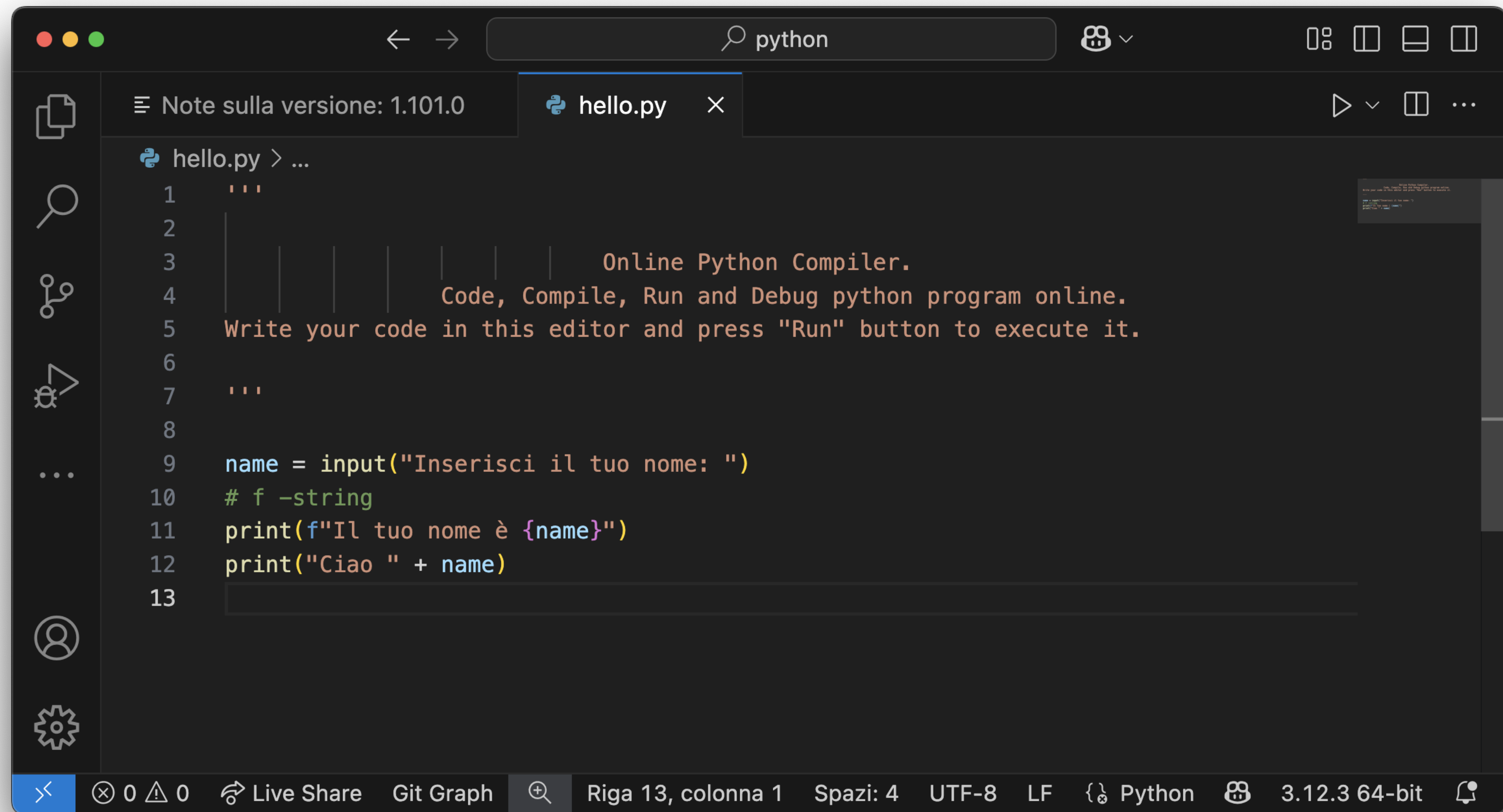
- Una **costante** è un valore che non cambia durante l'esecuzione del programma.

main.py

```
1  PI = 3.14159
2  MAX_UTENTI = 100
3  NOME_APP = "GestioneFile"
4
```

Primo esercizio

Riferiamo la nostra identità al programma...



The screenshot shows a web-based Python IDE. The browser's address bar contains 'python'. The IDE interface includes a sidebar on the left with icons for file explorer, search, source control, and settings. The top bar shows the current file 'hello.py' and a 'Run' button. The main editor area displays the following code:

```
1  """
2
3      Online Python Compiler.
4      Code, Compile, Run and Debug python program online.
5      Write your code in this editor and press "Run" button to execute it.
6
7  """
8
9  name = input("Inserisci il tuo nome: ")
10 # f-string
11 print(f"Il tuo nome è {name}")
12 print("Ciao " + name)
13
```

The status bar at the bottom provides additional information: 'Riga 13, colonna 1', 'Spazi: 4', 'UTF-8', 'LF', 'Python', '3.12.3 64-bit', and a notification bell icon.

Sintassi Python

- La sintassi di Python è stata progettata per essere il più vicina possibile al linguaggio naturale.
- L'assegnazione di variabili è diretta e intuitiva.
- La caratteristica più distintiva di Python è l'uso dell'indentazione per definire i blocchi di codice.

Lessico Python

Sintassi base

- **Assegnazione di variabili:**

- `x = 10 nome = "Alice" prezzo = 4.99 is_active = True`

Caratteristiche fondamentali:

- **Indentazione obbligatoria:** Python usa gli spazi invece delle graffe
- **Nessuna dichiarazione di tipo:** il tipo è determinato automaticamente
- **Case sensitive:** Nome e nome sono variabili diverse
- **Commenti:** utilizzare `#` per commenti su una riga

Lessico Python

Operatori

- **Operatori aritmetici:**
 - **$a + b$ # addizione**
 - **$a - b$ # sottrazione**
 - **$a * b$ # moltiplicazione**
 - **a / b # divisione**
 - **$a ** b$ # elevamento a potenza**
 - **$a \% b$ # modulo (resto della divisione)**

Lessico Python

Operatori

- **Operatori logici:**
 - True and False # AND logico
 - True or False # OR logico
 - not True # NOT logico
- **Operatori di confronto:**
 - a == b # uguale
 - a != b # diverso
 - a > b # maggiore
 - a >= b # maggiore o uguale

Secondo esercizio

Dato un numero intero come input, il programma deve restituire prima il suo precedente, poi il suo successivo, infine il doppio del numero inserito.

Terzo esercizio

Data la seguente formula:

$$C = \frac{5}{9} (F - 32)$$

Dove:

- C: gradi Celsius;
- F: gradi Fahrenheit

Si scrivi un programma che, preso in input la temperatura in gradi Fahrenheit, faccia la conversione in gradi Celsius

Esempio: 77 gradi Fahrenheit —> 25 gradi Celsius

Lessico Python

Strutture di controllo – Sintassi condizionale

```
eta = 18
```

```
if eta >= 18:
```

```
    print("Maggiorenne")
```

```
    print("Può votare")
```

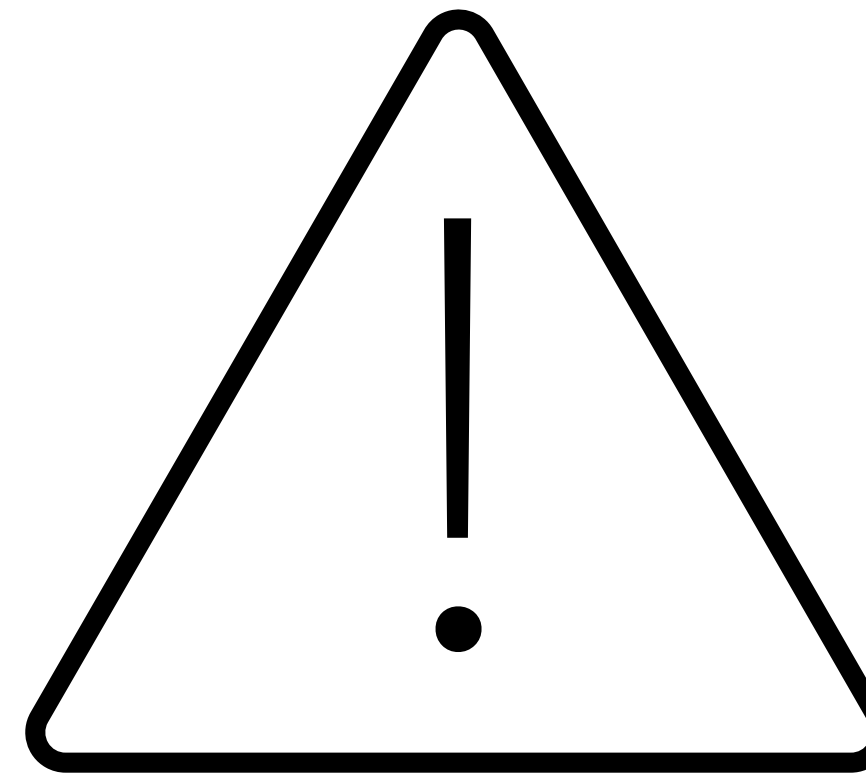
```
else:
```

```
    print("Minorenne")
```

Lessico Python

Strutture di controllo – Sintassi condizionale multipla

```
voto = 85  
  
if voto >= 90:  
    print("Ottimo")  
  
elif voto >= 70:  
    print("Buono")  
  
else:  
    print("Sufficiente")
```



Indentazione!

Lessico Python

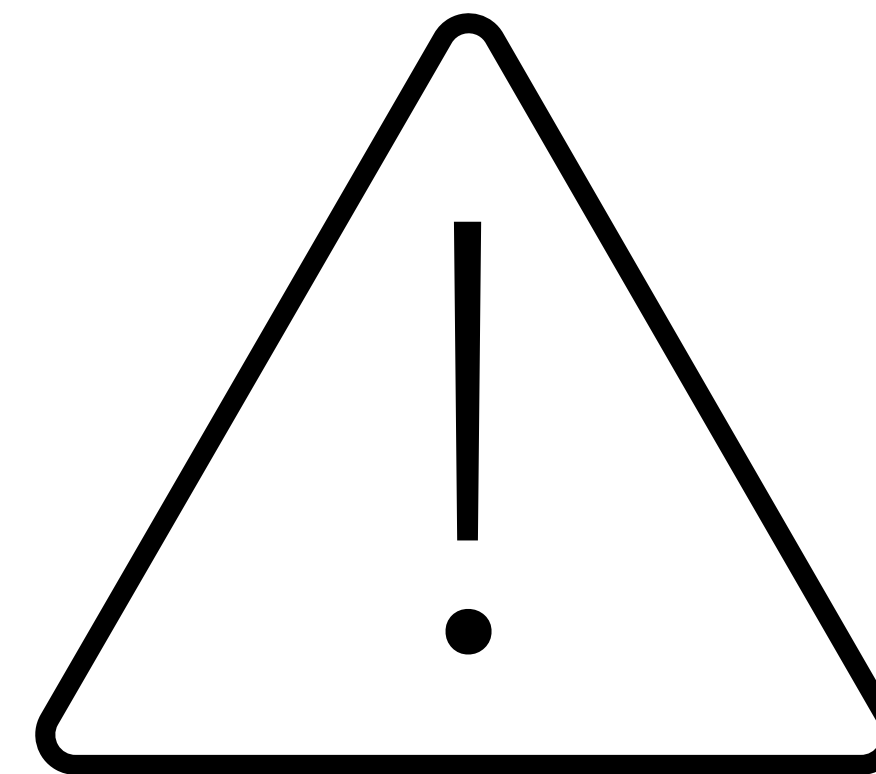
Strutture di controllo – Ciclo for

Ciclo con range

```
for i in range(5):  
    print(f"Numero: {i}") # Output: 0, 1, 2, 3, 4
```

Ciclo su lista

```
frutti = ["mela", "banana", "pera"]  
  
for frutto in frutti:  
    print(f"Mi piace la {frutto}")
```



Indentazione!

Lessico Python

Strutture di controllo – Ciclo while

```
x = 0
```

```
while x < 5:
```

```
    print(f"x vale {x}")
```

```
    x += 1 # equivalente a x = x + 1
```

Controllo:

break: esce dal ciclo

continue: salta all'iterazione successiva

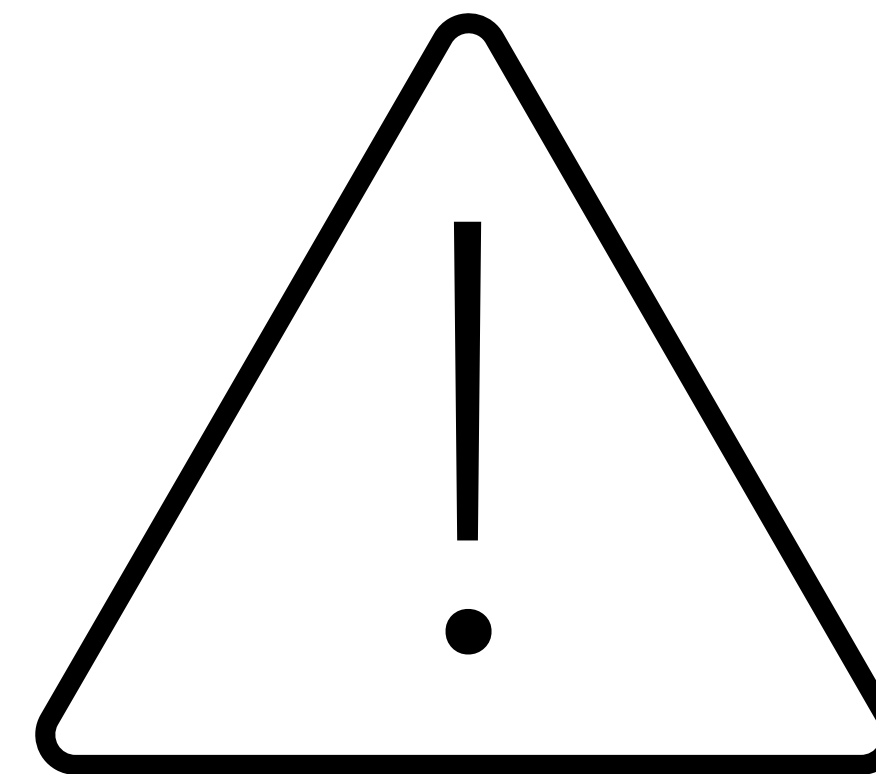
```
numeri = [1, 2, 3, 4, 5]
```

```
for num in numeri:
```

```
    if num == 3:
```

```
        continue # salta il 3
```

```
    print(num)
```



Indentazione!

Quarto esercizio

Scrivi un programma che:

Chieda all'utente quanti numeri desidera inserire.

Per ciascun numero:

1. Se è positivo e pari, stampa "Positivo e pari".
2. Se è positivo e dispari, stampa "Positivo e dispari".
3. Se è zero, stampa "È zero".
4. Se è negativo, stampa "Numero negativo".

Al termine, stampa:

Il conteggio dei numeri positivi, negativi e pari.
Il totale e la media dei numeri inseriti.